

Opia: OpenCode-Compatible Architecture Specification

Opia is a next-generation, extensible AI CLI agent. It combines an immersive, ai-shell-inspired developer experience with a robust, provider-agnostic backend.

This document outlines the complete system architecture, divided into two primary domains:

1. **Authentication & Provider System:** The core routing and LLM multi-provider framework.
2. **Tooling & Plugin Ecosystem:** The execution environment supporting MCP, Codex, Claude Code, and OpenClaw extensions.

Part 1: Authentication & Provider System

1.1 Goal

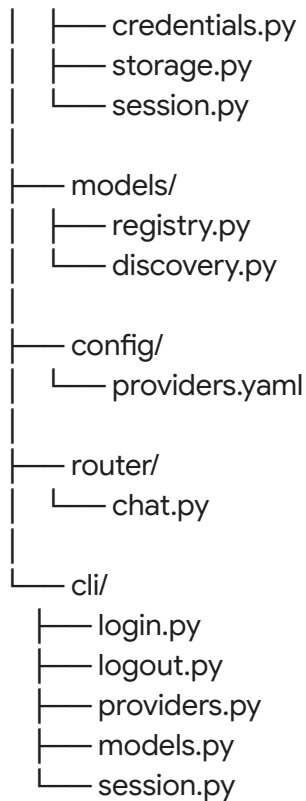
Implement an OpenCode-inspired authentication and provider architecture within Opia. The user experience must feel seamless while remaining completely provider-agnostic, modular, and extensible.

Core Capabilities:

- Multi-provider authentication & dynamic model discovery
- Secure credential storage & environment variable fallback
- Unified request routing & provider abstraction
- Session management & token refresh for OAuth

1.2 Architecture Directory Structure

```
opia/  
├── providers/  
│   ├── base.py  
│   ├── openai.py  
│   ├── anthropic.py  
│   ├── groq.py  
│   ├── openrouter.py  
│   ├── mistral.py  
│   ├── gemini.py  
│   ├── fireworks.py  
│   ├── together.py  
│   ├── deepseek.py  
│   └── registry.py  
├── auth/  
│   └── manager.py
```



1.3 Core Provider Interface

Every provider must inherit from a common base interface to guarantee provider-independent routing, consistent model access, and simple plugin integration.

```
class BaseProvider:
    name: str

    async def authenticate(self):
        ...
    async def list_models(self):
        ...
    async def chat(self, messages):
        ...
    async def embeddings(self, input):
        ...
    async def validate_credentials(self):
        ...
    async def refresh_token(self):
        ...
```

1.4 Authentication & Secure Storage

API keys must **never** be stored in plaintext. `auth/storage.py` handles credential resolution in the following order:

1. **System Keychain** (macOS Keychain, Linux Secret Service/Keyring, Windows Credential Manager)
2. **Encrypted Local Storage** (Fallback: `~/apia/credentials.enc` via AES-256-GCM encryption)
3. **Environment Variables** (e.g., `OPENAI_API_KEY`, `GROQ_API_KEY`)

When a user logs in (e.g., `apia login groq`), Opia immediately validates the key against the provider's `/models` endpoint. **Invalid credentials are never persisted.**

1.5 Session & Model Management

- **Active Provider:** Set via `apia provider use groq`. All requests default to this provider unless overridden via CLI flags (e.g., `apia chat --provider openai`).
- **Model Discovery:** `apia models list` dynamically fetches and caches available models (e.g., `gpt-5`, `llama-4`, `claude-opus`), preserving provider-specific metadata.
- **Session Tracking:** `auth/session.py` tracks the active provider, model, auth status, and timestamps.

1.6 Unified Request Router

Consumers (CLI commands or plugins) **never** interact with providers directly. All inference requests pass through `router/chat.py`, which resolves the provider, loads credentials, selects the model, normalizes responses, and handles retries.

```
await router.chat(  
    provider="groq",  
    model="llama-4",  
    messages=messages,  
)
```

Part 2: Tooling & Plugin Ecosystem

2.1 Goal

Opia is an extensible AI agent platform. Its architecture must support external tools, plugins, skills, and execution environments through a unified abstraction layer, comparable to OpenCode, Codex CLI, and Claude Code.

2.2 Plugin Manager Architecture

The plugin system is modular, allowing new capabilities to be added without modifying the Opia core.

```

opia/
├── plugins/
│   ├── manager.py    # Discovery, installation, permissions
│   ├── registry.py
│   ├── loader.py     # Runtime loading/unloading
│   ├── sandbox.py
│   └── adapters/    # Translation adapters for external ecosystems
│       ├── mcp.py
│       ├── codex.py
│       ├── claude.py
│       ├── openclaw.py
│       └── native.py

```

2.3 Supported Plugin Ecosystems & Adapters

Opia provides native support or compatibility layers for industry-standard tool protocols:

- **MCP (Model Context Protocol):** Full support for local/remote MCP servers, skills, tools, streaming, and transport over stdio/HTTP/SSE.
- **Codex CLI:** Compatibility for file editing, shell commands, planning workflows, and multi-step tasks.
- **Claude Code:** Adapter layer to run Claude Code plugins/skills, file operations, and project understanding APIs without modification.
- **OpenClaw:** Support for native OpenClaw plugins, agent extensions, and workflow plugins.

2.4 Native Tool System

All tools—whether built-in or loaded via an adapter—conform to a common abstraction:

```

class BaseTool:
    name: str
    description: str

    async def execute(self, **kwargs):
        ...

```

2.5 Core Built-in Tools

Opia includes first-party implementations of essential agent tools:

- **Filesystem:** Read, write, patch, move, search, and watch files/directories.
- **Terminal:** Execute shell commands, stream output, background jobs, and environment isolation.

- **Search:** Fast code search, semantic search, workspace indexing, and symbol lookup.
- **Git:** Status, diff, commit, branch, merge, rebase, and patch generation.
- **Web:** HTTP requests, web search, API exploration, and browser automation adapters.
- **Editor:** Multi-file refactoring, code formatting, diagnostics, and symbol renaming.
- **Task Management:** Planning, checkpoints, task graphs, and progress tracking.

2.6 Capability Detection & Security Model

Plugins execute with explicit, auditable permissions. Each plugin exposes metadata to allow Opia to automatically determine which tools can satisfy a user's request.

```
PluginInfo(
    name="filesystem",
    version="1.0.0",
    supports_streaming=True,
    permissions=[
        "filesystem.read",
        "filesystem.write",
    ],
)
```

Part 3: Success Criteria Checklist

Provider System Checklist

- ☐ Multiple AI providers operate through a unified interface.
- ☐ Credentials are encrypted and securely stored.
- ☐ Environment variables are automatically supported.
- ☐ Login and logout workflows operate correctly (immediate validation).
- ☐ Active provider switching and per-request overrides function reliably.
- ☐ Dynamic model discovery works across providers.
- ☐ A unified router handles every request.
- ☐ OAuth-based token refresh is supported in the base interface.

Tooling & Plugin Checklist

- ☐ MCP servers and skills work natively.
- ☐ Codex CLI, Claude Code, and OpenClaw plugins are supported via adapters.
- ☐ Plugins can be installed/hot-loaded without modifying the Opia core.
- ☐ Built-in tools provide Filesystem, Terminal, Git, Web, and Editor capabilities.
- ☐ All tools share the BaseTool execution interface.
- ☐ Capability detection (metadata) and permission management function correctly.